

METRO Integration Reference Architecture

INTRODUCTION	3
MOTIVATION.....	3
SCOPE	3
TARGET AUDIENCE	3
INTEGRATION REFERENCE.....	5
GLOSSARY.....	5
INTEGRATION LANDSCAPE	6
API BASED PATTERNS.....	7
<i>API Gateway</i>	7
<i>CRUD (Create, Read, Update, Delete)</i>	9
<i>Facade</i>	9
<i>Aggregation</i>	9
<i>Authentication and Authorization</i>	10
<i>Caching</i>	10
<i>Partial Response (Field Selection)</i>	10
<i>gRPC</i>	12
<i>Websockets</i>	12
MESSAGE BASED PATTERNS	13
<i>Publish-Subscribe</i>	14
<i>Point-to-Point (Queues):</i>	15
DATA INTEGRATION PATTERNS	15
<i>ETL (Extract, Transform, Load)</i>	15
<i>ELT (Extract, Load, Transform)</i>	15
<i>Data Synchronization</i>	16
MIDDLEWARE-BASED INTEGRATION	16
<i>Broker</i>	16
<i>EDI (Electronic Data Interchange)</i>	16
ANTI-PATTERNS.....	17
<i>Shared Database</i>	17
<i>File-based data transfer</i>	17
PRESCRIPTIVE GUIDANCE	18
AS-IS SOLUTION SELECTION BY DATA OBJECT	18
INTEGRATION DOS	22
INTEGRATION DON'TS	23
DOCUMENTATION IN LEANIX	ERROR! BOOKMARK NOT DEFINED.

Introduction

An integration reference architecture is an architecture definition at a higher level of abstraction, where the elements and relations provide a template for concrete integration designs. The intent is to define standardized interfaces and integration mechanisms, to use them in a uniform manner so that we tackle the integration pain-points encountered across METRO.

We structure this in three main sections, starting with a definition of the fundamental integration concepts and patterns used across the reference architecture, an architecture overview for all relevant integration contexts where we define the building blocks with their interactions, and ending with a section of prescriptive guidance where we provide more practical advice about what generally should be followed and what should be avoided.

Motivation

In our complex application landscape, we have a need for efficient and effective integration between solutions and IT components. Traditionally, file-based and ESB-based integrations have been the go-to strategies for connecting our systems. Since these, while reliable at the time, have shown limitations in terms of scalability, flexibility, and ease of use, we strive for a more modern integration approach and provide an integration reference architecture in this document.

Scope

This reference architecture is applicable to integration between METRO.digital solutions, MCC country local applications, METRO legal entities applications, METRO service companies applications and external entities applications (SaaS providers, Customers, Suppliers, Logistic Service Providers, Government).

It is NOT applicable to integrations inside a solution/application (at the IT component level).

It is NOT applicable to integrations by [process orchestration](#) (Workflow, RPA).

It is NOT applicable to integrations by [data virtualization](#).

Target Audience

There are multiple roles that are interested in the integration reference architecture:

- **Technical Enterprise Architects** will use this document to verify that the right integration approach has been chosen during architecture reviews. They will point out improvements based on this integration reference architecture
- **Solution Architects** (from METRO.digital, MCC countries, Metro legal entities, METRO service companies) will use this document to analyse options and finally choose the right integration approach between their solutions/applications and other solutions/applications (corporate or local) or external systems. This document is

giving them guidance on which data objects to consume from where, which integration type to adopt and what pitfalls to avoid for their specific context.

- **Tech Leads** can use this document for prescriptive guidance on new integrations implementation and assessing compliance of their current integration with best practices.

Integration Reference

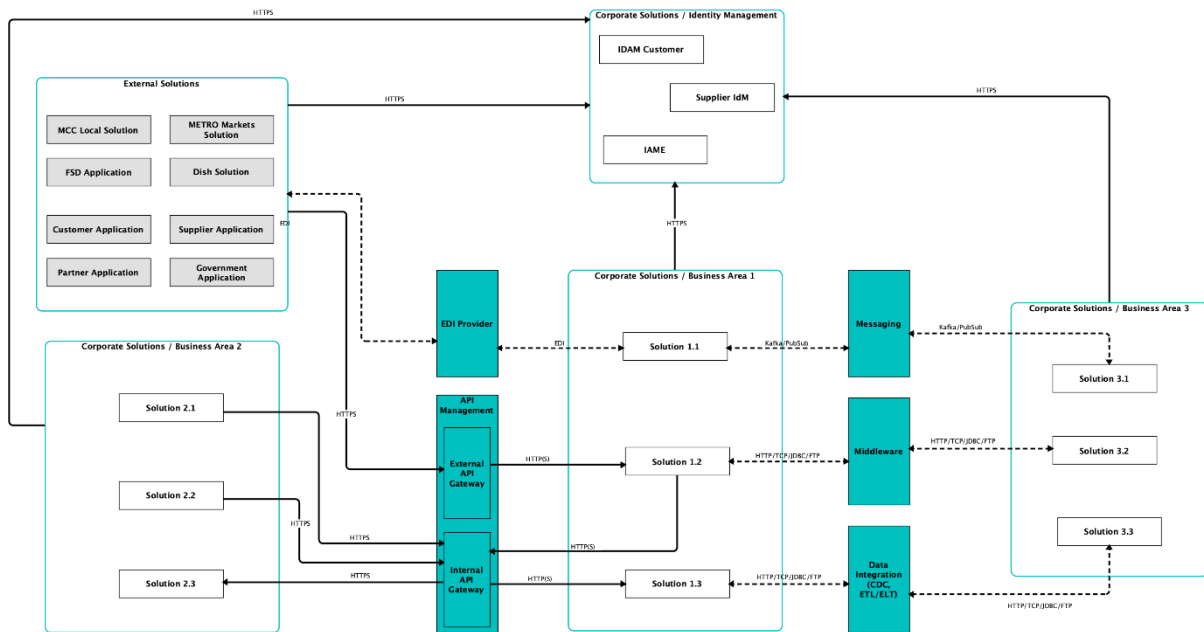
Glossary

Term	Description
Integration Platform	A group of solutions and technologies providing as technical capability the integration between counterparties.
Counterparty	METRO Solutions and IT Components taking part in one or many integrations.
Interface	Realization of an integration contract between two or many counterparties (see LeanIX).
Provider	A Counterparty that provides data and/or messages
Consumer	A Counterparty that consumes data and/or messages
Message	A structured data item transmitted from Provider to Consumer. It generally has two parts, a header describing data being transmitted, and a payload as data being transmitted. Messages can be of various types: commands, queries, events or responses.
Integration type	A method used to combine certain software components to enable integration for specific requirements and messages.
API-based integration	An integration type where APIs serve as the interface through which systems interact, allowing them to access data and functionalities based on a predefined contract between API Provider and API Consumer.
Point-to-Point based integration	An integration type where there is only one Provider and one Consumer, while the communication can be direct or through an intermediary component (ex. queue).
Publish/Subscribe based integration	An integration type where Providers generate messages and publish them to one or more channels, and Consumers express interest in specific topics and receive messages that are published to those channels.
Middleware based integration	An integration type where middleware serves as an intermediary layer between Provider s and Consumers, providing a set of services for integrating and coordinating interactions between them. One example is Extract Transform Load (ETL) tools.
Change Data Capture based integration	An integration type used to capture and propagate changes made to data in a database or data source in real-time or near-real-time.
EDI based integration	An integration type based on Electronic Data Interchange standard to facilitate the exchange of business documents and transactions between different organizations or trading partners.

API Leveling	Classification of APIs by levels of access for different consumers.
External entity	Non-METRO legal entities interested in integrating and exchanging data with METRO: Customers, Suppliers, Partners, Logistic Service Providers, Government.
RPA	Robotic Process Automation (RPA) is a technology that uses software robots or 'bots' to automate highly repetitive, routine tasks that were previously performed by human workers. These tasks typically involve interacting with digital systems and software, such as data entry, transaction processing, or responding to simple customer service queries.
TLS	Transport Layer Security (TLS) is a cryptographic protocol designed to provide secure communication over a computer network. It is widely used to secure data transmitted over the internet, ensuring privacy and data integrity between communicating applications.
OAuth 2	OAuth 2 (Open Authorization) is an authorization framework that allows third-party applications to obtain limited access to a user's resources without exposing their credentials. It is widely used to grant websites or applications limited access to user information without compromising security.

Integration Landscape

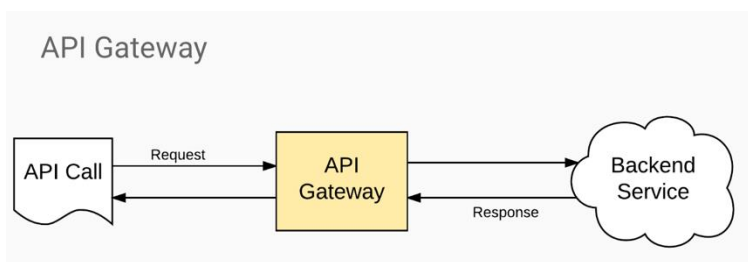
To transition from the current heterogeneous integration landscape, we introduce the concept of an Integration Platform that acts as an intermediate layer between Provider and Consumer solutions and applications. This Integration Platform is meant to be capable of fulfilling all METRO integration needs by encapsulating all necessary integration patterns with their corresponding technologies. Subsequently, we describe these patterns and specify when they should be utilized for a given context.



API based patterns

API Gateway

API-based integrations should be used for **request-response, synchronous** calls from many to one system. It contrasts with traditional point-to-point integration by emphasizing the creation of a reusable API which is accessed by many systems through an API Gateway, acting as a proxy between the consumer and producer service.



The API gateways used will provide the following capabilities and should be selected according to the use-case:

Function	Benefit
Routing	* Simplifies client interactions by providing a single point of access. * Enables backend changes without impacting clients.
Load Balancing	* Optimizes performance by distributing requests. * Prevents backend service overload.

	* Improves service availability.
Authentication	* Ensures only authorized users/systems access APIs. * Protects sensitive data. * Maintains data privacy.
Authorization	* Controls granular access to APIs based on permissions. * Enforces security policies. * Helps meet compliance requirements.
Rate Limiting & Caching	* Protects backend services from overload. * Improves resource utilization. * Enhances application responsiveness.
Monitoring & Analytics	* Provides insights into API usage patterns. * Facilitates troubleshooting. * Aids in capacity planning and optimization.
Transformation	* Translates data formats and protocols (e.g., JSON to XML). * Ensures compatibility between clients and backend systems.

Best Practices:

- Secure APIs with authentication (OAuth2) and authorization
- Implement versioning
- Make definitions available in discovery URLs
- Use mutual TLS with external facing APIs that are used for SaaS-to-SaaS providers
- Implement complete error code standards
- Developed **RESTful APIs** can be graded using the [Richardson Maturity Model](#).

Use Cases:

- Enabling mobile apps, external partners, or self-service portals to access master data.
- Integrate legacy systems by providing a facade translation layer to the legacy system interface.

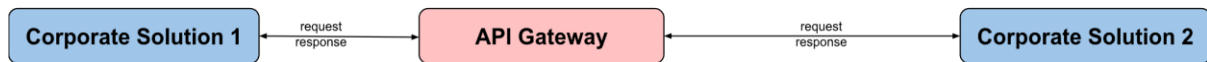
Stateless, request-response APIs over HTTP/HTTPS protocol fall into 2 architectural styles: RESTful and GraphQL

RESTful APIs follow a resource-oriented approach, using standard HTTP methods to interact with data. They are known for their scalability and simplicity, making them suitable for a wide range of applications.

GraphQL APIs allow clients to request specific data fields and structures, reducing over-fetching and under-fetching of data. This flexibility and efficiency are advantageous for complex, data-driven applications and mobile clients with limited bandwidth.

Developers can choose between RESTful and GraphQL based on their project requirements and preferences.

CRUD (Create, Read, Update, Delete)

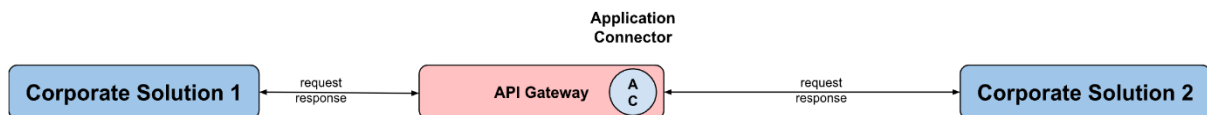


The API Gateway overlaying the data source acts as a simple proxy to the source systems API.

The fundamental pattern for interacting with resources. HTTP verbs map to these actions:

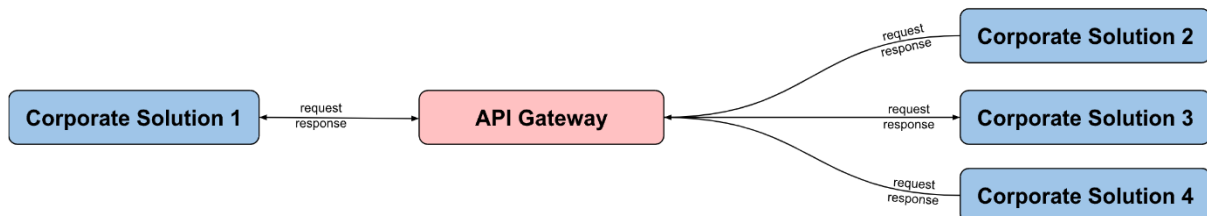
- POST: Create a new resource
- GET: Read an existing resource
- PUT: Update an existing resource (replaces entirely)
- PATCH: Partially modify an existing resource
- DELETE: Delete a resource

Facade



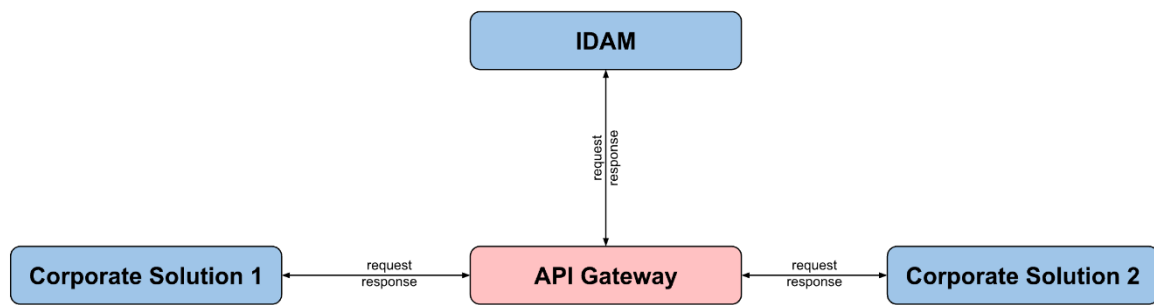
Simplifies interactions with a complex backend system, providing a unified API to corporate solutions own API. Typically achieved using vendor application connector or generic translation services applied in the API gateway. e.g. SOAP connector, Salesforce connector

Aggregation



Combines data from multiple sources into a single API response, saving clients from having to make multiple requests.

Authentication and Authorization

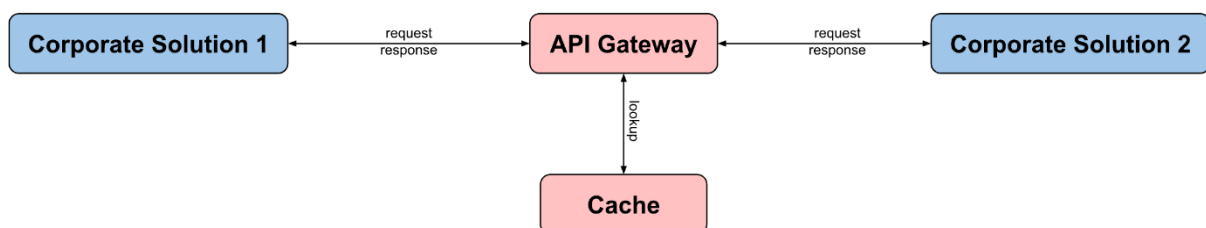


Mandatory pattern to be included across all HTTPS integrations to align with security best practices.

- OAuth2 - Secure delegated access for third-party applications.
- JWT (JSON Web Tokens) - Compact tokens for authentication and authorization.
- Basic Authentication - Simple but less secure (consider only with HTTPS).

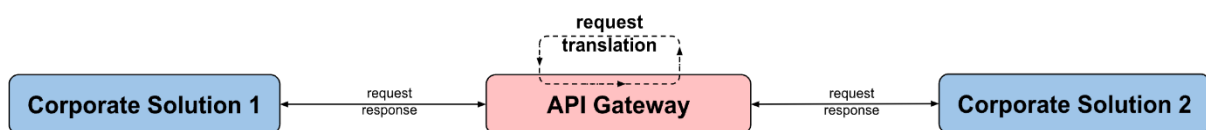
Authorisation typically integrates with overarching Identity Provider (IDP), here IDAM.

Caching



Leverages HTTP headers like ETag and Cache-Control to improve performance by avoiding the transfer of redundant data.

Partial Response (Field Selection)



Allows clients to specify the exact fields they need in a response, reducing data transfer.

Best Practices:

- **Resource Naming:** Use nouns to represent resources and clear hierarchical structures in URLs.
- **HTTP Verbs:** Employ the appropriate HTTP verb for each action (GET for retrieval, POST for creation, PUT for updates, DELETE for removal).
- **Status Codes:** Utilize standard HTTP status codes to convey the outcome of requests.

- **Error Handling:** Provide meaningful error messages and appropriate status codes for errors.
- **Versioning:** Implement versioning to maintain backward compatibility as APIs evolve.
- **Security:** Employ authentication and authorization mechanisms to secure APIs.
- **Documentation:** Maintain comprehensive and up-to-date API documentation.

Use Cases:

- **Web Applications:** Building interactive and dynamic web applications that fetch and update data from servers.
- **Mobile Applications:** Powering mobile apps that rely on backend services for data and functionality.
- **Microservices:** Enabling communication and data exchange between microservices in a distributed architecture.
- **Third-Party Integrations:** Allowing external applications or services to interact with your system.
- **Public APIs:** Exposing data and services to a wider audience, potentially generating revenue or fostering innovation.

Best Practices:

- **Schema Design:** Design your GraphQL schema using clear types and relationships to represent your data model accurately.
- **Field Level Authorization:** Implement fine-grained authorization at the field level to ensure data security and privacy.
- **Pagination:** Design pagination mechanisms to handle large datasets efficiently, preventing performance issues.
- **Error Handling:** Provide informative error messages with relevant codes and details to assist client-side debugging.
- **Performance Optimization:** Employ techniques like caching, batching, and query optimization to improve GraphQL API performance.
- **Monitoring and Logging:** Monitor query performance, track usage patterns, and log errors for continuous improvement.

Use Cases:

- **Mobile Applications:** GraphQL's ability to fetch precise data in a single request makes it ideal for mobile apps with limited bandwidth.
- **Single-Page Applications (SPAs):** GraphQL simplifies data fetching for SPAs, reducing network round trips and improving user experience.
- **Complex UIs:** Applications with complex UIs can benefit from GraphQL's flexibility, allowing them to request and display data efficiently.
- **Content Management Systems (CMSs):** GraphQL can empower content editors with a flexible way to query and retrieve content for various platforms.

Example: M.Shop, Customer self-service

gRPC

gRPC is a high-performance open-source remote procedure call (RPC) framework developed by Google. It enables efficient communication between services in a distributed system, leveraging the power of Protocol Buffers for data serialization and HTTP/2 for transport. This results in faster, more compact message exchange compared to traditional RESTful APIs.

Best practices:

- **Interface Design:** Design your gRPC services with well-defined interfaces using Protocol Buffers, considering versioning for future compatibility.
- **Error Handling:** Implement robust error handling mechanisms, leveraging gRPC's built-in error codes and custom error messages for detailed feedback.
- **Streaming:** Utilize gRPC's streaming capabilities for efficient handling of large datasets or real-time data streams.
- **Security:** Implement appropriate authentication and authorization measures to secure your gRPC services, such as TLS encryption and token-based authentication.
- **Monitoring:** Monitor the performance of your gRPC services to identify bottlenecks and ensure optimal performance.

Use cases:

- **Microservices Communication:** gRPC is ideal for communication between microservices in a distributed architecture due to its efficiency and performance.
- **Internal APIs:** gRPC is a good choice for internal APIs where performance and efficiency are prioritized over interoperability with external systems.
- **Client-Server Applications:** gRPC is well-suited for client-server applications where a high-performance, bidirectional communication channel is required.
- **Real-time Systems:** gRPC's streaming capabilities make it an excellent choice for real-time systems that require the exchange of data streams.

Websockets

WebSockets provide a full-duplex communication channel over a single TCP connection, enabling real-time, bidirectional data exchange between a client (e.g., a web browser) and a server. This persistent connection eliminates the need for frequent polling requests, resulting in lower latency and more efficient communication compared to traditional HTTP-based approaches. WebSockets find extensive use in applications demanding real-time updates, such as chat apps, collaborative tools, live dashboards, and online gaming.

By keeping connections open, WebSockets do however block resources on the server application. Scaling [WebSockets is not trivial](#) and fallbacks might need to be implemented since company proxy servers might block their connections. Hence, while there might be use cases for WebSockets, this approach and technology should be used sparingly, and only when requirements rightfully justify it.

Best Practices:

- **Connection Management:** Implement robust connection management to handle connection establishment, maintenance, and graceful disconnection.
- **Error Handling and Retries:** Develop strategies for handling connection errors and implementing automatic reconnection mechanisms.
- **Security:** Prioritize security by utilizing WebSocket Secure (WSS) for encryption and implementing appropriate authentication and authorization measures.
- **Message Framing:** Clearly define message framing protocols to ensure reliable parsing of messages between client and server.
- **Scalability:** Design WebSocket servers for scalability to accommodate a growing number of concurrent connections.
- **Rate Limiting:** Implement rate limiting to prevent abuse and protect against denial-of-service attacks.

Use Cases:

- **Real-time Updates:** Push real-time notifications, updates, or data changes to clients in applications like stock tickers, news feeds, and social media platforms.
- **Collaboration:** Enable seamless real-time collaboration in document editing, shared whiteboards, and multi-user applications.
- **Chat and Messaging:** Facilitate instant messaging and chat functionalities with minimal latency.
- **IoT and Telemetry:** Stream telemetry data from IoT devices or sensors to central systems for monitoring and analysis.
- **Gaming:** Power real-time interactions and updates in online multiplayer games.
- **Live Streaming:** Deliver live audio or video streams with minimal delay.

Message based patterns

Message-based integration should be used when you prioritize **asynchronous** communication, **loose coupling**, and **reliable message delivery**.

Well-suited for scenarios where systems need to operate independently, where you expect bursts of incoming traffic, or where guaranteed message processing is essential.

Primary use (and main difference from Middleware-based integration) is for event-driven architectures, where systems react to events generated by other systems, or microservice architectures where numerous small services need to communicate without tight dependencies. Additionally, if systems need to scale individually or have varying availability, message-based integration allows for better resilience and flexibility.

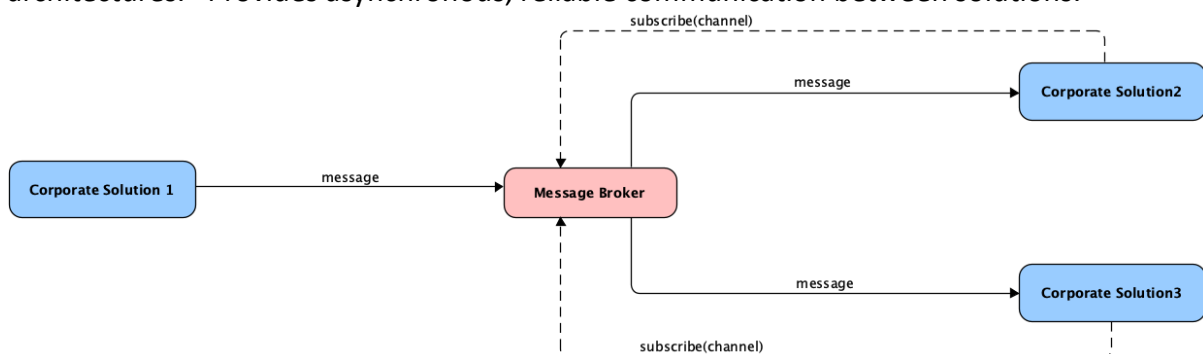
Message based integrations are realized using **message brokers**, such as Kafka, RabbitMQ, Google Pub/Sub.

Function	Benefit
----------	---------

Asynchronous Messaging	* Decouples systems, allowing them to operate independently. * Improves responsiveness by avoiding blocking operations. * Handles bursts of incoming messages effectively.
Publish/Subscribe (Pub/Sub)	* Distributes messages efficiently to multiple interested subscribers. * Facilitates event-driven architecture and real-time updates. * Enables loose coupling between systems.
Message Queuing	* Provides reliable delivery guarantees. * Ensures messages are processed even in case of system failures. * Helps smooth out peak traffic demands.
Persistence	* Stores messages for durability (sometimes configurable). * Prevents data loss in case of broker outages. * Allows for replaying messages, if needed.
Message Transformations	* Modifies messages in transit (e.g., format conversion, filtering). * Ensures compatibility between message producers and consumers. * Enriches messages with additional data.
Monitoring	* Provides insights into message flow, system health, and performance. * Assists in troubleshooting and capacity optimization.

Publish-Subscribe

Decoupled asynchronous and reliable communication typically used to create event-driven architectures. Provides asynchronous, reliable communication between solutions.



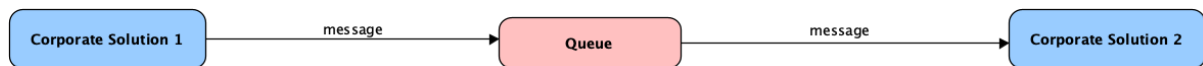
Best Practice: One-to-many messaging. Where message reliability is needed.

Use Case: Triggering order fulfillment processes when an order is placed or broadcasting promotional offers to subscribers.

Example: M.SHOP is providing update events on order objects in a feed. Clients like M.FULFILL PICK & PACK or M.TRANSPORT consume these events to trigger a reaction in their services (adding picklists to picking waves or adjusting transport tour information, for instance).

Point-to-Point (Queues):

Reliable one-to-one communication.



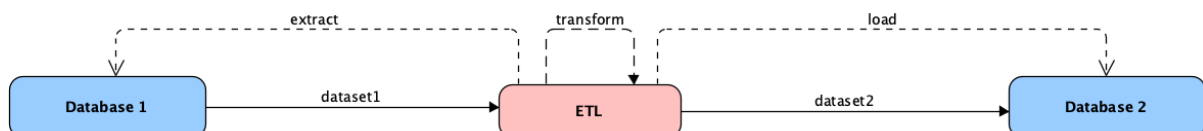
Best Practice: Use for load balancing and ensuring order of processing. Mandatory to direct through message broker to provide visibility and monitoring.

Use Case: Distributing tasks to multiple back-end workers, sequencing steps in a complex workflow.

Data Integration patterns

ETL (Extract, Transform, Load)

Extracting data from sources, transforming it into a target schema, and loading it into data warehouses/data lakes.



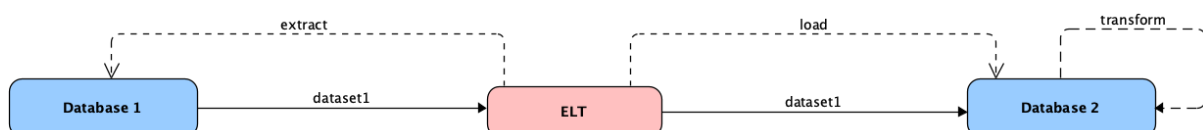
Best Practice: Define clear data lineage, implement data quality checks. Include error handling

Use Case: Copying data into centralized data repository for reporting and analytics.

Example: Data exports to DANA

ELT (Extract, Load, Transform)

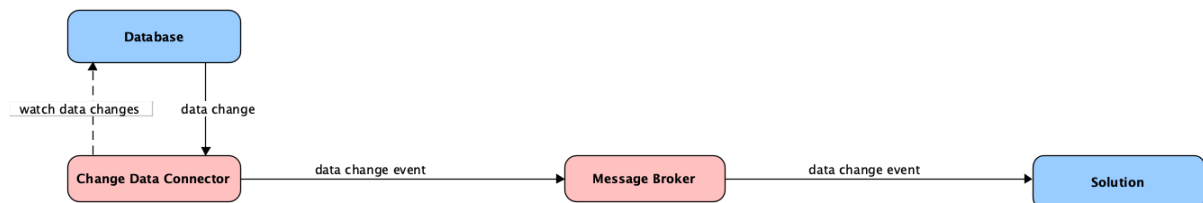
Leverages the processing power of the target data warehouse/lake to perform transformations.



Best Practice: Suitable for large volumes of data and flexible, exploratory data analysis.

Use Case: Building machine learning models that work on raw or semi-structured data.
Used in DANA (BQ)

Data Synchronization



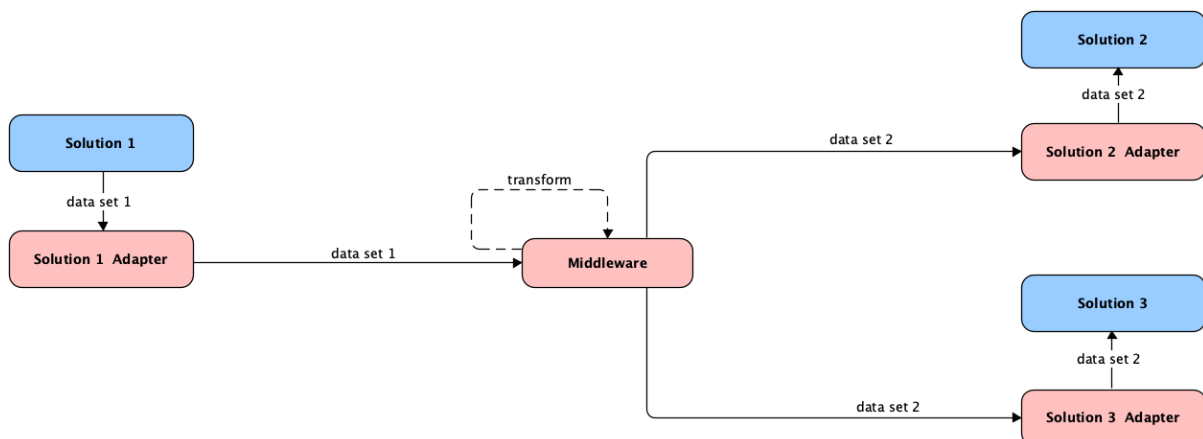
Use Case: Used for large scale migrations across clouds where synchronization of large databases is needed for a limited time. Can be used to keep database changes in sync with external systems, however this pattern is discouraged.

Ex. used in METRO.digital for updating M.Stock - MMS Store sync'ing events

Middleware-based Integration

Broker

Decoupled communication between components via a broker component. Additional tasks like filtering, transformation or ensuring quality of service can be performed.

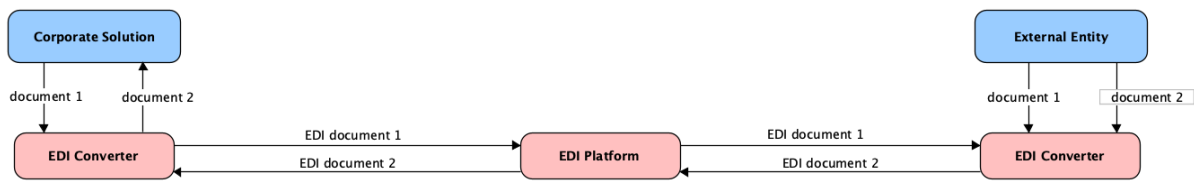


Best Practice: Suitable for hub-and-spoke communication

Use Case: Publishing messages in different formats to various consumers. c.f. MIP]
[Large finance transformations

EDI (Electronic Data Interchange)

Machine-to-machine documents exchange using a standard format agreed by both parties.



Best Practice: Suitable for communication with external entities: Supplier, Logistic Service Provider, Government, Banking, etc.

Use Case: Transferring Purchase Orders, Advice Notes, Invoices, Finance Transactions.

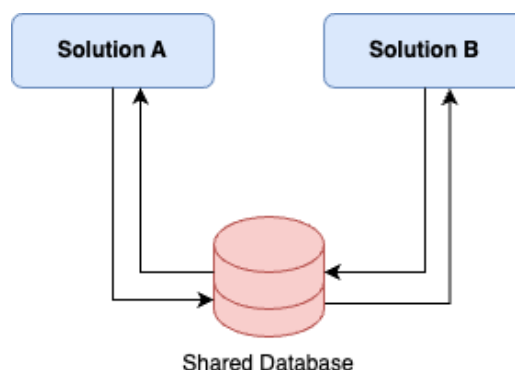
Anti-patterns

The following integration types constitute **anti-patterns** and must be avoided at all costs for any new integration, while the existing integrations using these should be considered for future refactoring as defined in this document.

Shared Database

Integrate multiple solutions via a shared database through JDBC/ODBC, DB Link. This type of integration comes with various drawbacks:

- **Strong dependency on database level:** one solution cannot change their internal database schema independently leading to alignment overhead and increased maintenance effort.
- **Consistency issues** can be the result of multiple solutions storing data in different formats, structure, or update frequency.
- **Lock-in on database technology:** the underlying database technology cannot easily be changed because alignment in multiple solutions is needed.
- **Decommissioning one solution is impossible without modifying the others significantly.**



File-based data transfer

Data is exchanged between two solutions via flat files, e.g. CSV or XML files. Both solutions align on a storage location to which a provider uploads files (usually via SFTP) and a consumer downloads them from.

File-based data transfer has several weaknesses:

- **Complexity and error-proneness** in the data integration
- **Reliability** issues due to complex error handling and data validation
- **Scalability**: Limited concurrency due to file corruption risks
- **Weak security** because in most cases, basic auth is chosen over an Identity-Provider-based solution

Use Cases

- transfer of large amounts of data in batches

Best Practices

- Use a modern approach using Cloud Storage and Pub/Sub notification to mitigate some of the risks in error-proneness and security areas
- Use a Facade API pattern to migrate legacy applications. Facade can translate a RESTful request into, say, a SOAP request in legacy schema

Prescriptive Guidance

As-Is Solution selection by Data Object

[Overview of data objects and solution owning the objects in LeanIX](#)

The following table gives guidance on which solution to integrate for a given Data Object in purpose. The table is based on the current state of the application landscape. Before development, the final call whether it is the right solution is made by Process Architects and Technical Enterprise Architects.

Data Object	Context	Source Solution	System of Records
Article Data	Article data for corporate solutions	M.Article Cache	MMS HO/Master Data Article
	Article grouping	M.LEADING ARTICLE	
	New Article listing, Golden record Article, listing & data processing automation	M.PIM	
	Article logistic conditions	MDLS	
	Article data, Product Information, Pricing combined exposed for digital channels	M.COMMERCIAL DATA	
Product Information	Product Information golden record	M.PIM	MMS HO/Master Data Article
	Product Information for own brand (depending on country, local solution is used).	MISC	

	Product Information for corporate solutions	M.Article Cache	
	Article data and Product Information combined exposed for digital channels	M.COMMERCIAL DATA	
Buying Price	Buying Price for corporate solutions	M.Article Cache	MMS HO / Assortment Management
	Buying Price for local solutions	MMS Store	
	Buying price for ultra-fresh products.	MMS Store	
	Real Buying Price	MMS IC & SIS	
Supplier	Supplier	MDLS	MMS HO / Supplier Master Data
	Supplier for corporate solutions	M.Article Cache	
	Supplier data created	MMS HO / Supplier	
	Supplier for ultra-fresh products	MMS HO / UFM	
	Source of truth	Supplier Master data by SAP	
Supplier Conditions	Supplier Commercial Conditions	MCCM	MCCM
	Supplier Logistics Conditions for corporate solutions	M.Article Cache	MMS HO / Supplier Master Data
Supplier Contract	Goods Supplier Contracts	MCCM	MCCM
Producer	Producers of own brand articles	M.Quality Own Brand	M.Quality Own Brand
	Producers of own brand	MISC	MMS HO / Supplier Master Data
	Producers of non-own brand for corporate solutions	M.Article Cache	
	For non-own brand.	M.COMMERCIAL DATA	
Assortment	Assortment for C&C	MMS Store	MMS HO / Assortment Management
	Assortment for C&C for corporate solutions	M.Article Cache	
	Assortment for FSD & Trader	M.COMMERCIAL DATA	M.Assortment Planning
METRO Entity	Reporting purpose	<i>no solution</i>	Konzernorganigramm zum 30.09.2022.pdf
Head Office	Contract management, GNFR, Financial reporting, accounting, legal	MMS HO	

Fulfillment Center	Fulfillment Center basic information	M.FULFILL DEPOT MANAGEMENT	M.FULFILL DEPOT MANAGEMENT
	Fulfillment Center warehouse layout and picking locations	M.FULFILL DEPOT MANAGEMENT	
	Fulfillment Center all stores in a country	M.Article Cache	MMS HO / Supplier Master Data
	Fulfillment Center information about one particular store	MMS Store	
Central Platform	Central Platform (address, opening hours, building plans)	MDLS	MDLS
	Central Platform identified by supplier number	MMS DC	MMS DC
Purchase Order	Purchase Orders for indirect flow (depending on country)	MDLS	
	Purchase Orders for ultra-fresh	MMS HO / UFM	MMS HO / UFM
	Purchase Orders for indirect flow (depending on country)	MMS DC / Warehouse ERP	MMS DC
	Purchase Orders for Direct Store Delivery (DSD)	MMS Store	MMS Store
Supplier Invoice	Supplier Invoice (depending on country)	M.Supplier Invoice Control 1.0	M.Supplier Invoice Control 1.0
	Supplier invoice (depending on country)	M.Supplier Invoice Control 2.0	M.Supplier Invoice Control 2.0
Stock	Stock quantity information	M.STOCK	MMS Store, MMS DC, MDLS
	Stock value information (depending on country)	M.STOCK EVALUATION	M.STOCK EVALUATION
Selling List Price	Selling List price for C&C	M.Article Cache, MMS HO	MMS HO / Master Data Article
	Selling List Price for FSD	M.COMMERCIAL DATA	
Selling Promotion Price	Selling Promotion Price for C&C	MMS HO / Sales Price and Discount	MMS HO / Sales Price and Discount
	Selling Promotion Price for FSD	M.COMMERCIAL DATA	
Customer Price Contract	Customer individual prices	M.CIP	M.CIP

Prospect	Prospect (depending on country)	M.CUSTOMER PROFILE	M.CUSTOMER PROFILE
	Prospect (depending on country)	MCFM	
Person	Personal information about customers (depending on country)	M.CUSTOMER PROFILE	M.CUSTOMER PROFILE
	Personal information about customers (depending on country)	MCFM	
	Personal information about customers (depending on country)	M.CUSTOMER CACHE	
	Authentication/authorization information about customers	IDAM Customer	
Customer Account	Customer account (depending on country)	M.CUSTOMER PROFILE	M.CUSTOMER PROFILE
	Customer account (depending on country)	MCFM	
Customer Order	History of Customer Orders, placing Customer Orders	M.SHOP	M.SHOP
Customer Invoice	Historical invoices after retention time	M.INVOICE – Invoice	M.INVOICE – Archive
	Consolidated invoices	M.INVOICE - CONI	M.INVOICE - CONI
	Recent invoices before retention time	M.POS X / M.POS Air	M.POS X / M.POS Air
Employee	Employee HR - related info (home address, bank account, age, contract information)	One HR - Oracle HCM	SAP
Employee Identity Account	Employee authentication information	IAME	IAME
	Employee authorization information (positions, roles)	IAME	IAME

Ledger	Ledger	MGFS by SAP S/4 HANA	MGFS by SAP S/4 HANA
---------------	--------	----------------------	----------------------

Integration DOs

Rule	Impact
DO take data directly from source solutions that are Single Version of Truth for the targeted data objects.	Minimize data quality and data inconsistency issues, simplify the quality control, monitoring, reconciliation inter-team activities. Exceptions might be possible when there are data aggregation services exposed (ex. Evaluate, CDP).
DO enforce access leveling in your APIs	Every solution exposing APIs must document the consumers scope and restrict access accordingly: internal (only by the IT Components of that solution), corporate (only by METRO.digital solutions), public (by MCC, METRO entities and METRO service companies local applications)
DO use mechanisms to enforce interface contracts between producers and consumers	Use message schema-based validation and Consumer Driven Contracts to ensure compatibility on interface evolution. In case of APIs implement all relevant HTTP status codes in full compliance with their intent and semantics.
DO implement idempotency mechanisms (ex. deduplication) on the consumer solutions	Allow the same message/data set to be delivered more than once without side effects for the consumer solution, to increase reliability of the transfer and consistency of data.
DO use reliability mechanisms like timeouts, retries, circuit breaker (for RESTful services) or Dead Letter Queue (for event-driven services)	Make sure in case of failures or partial failures, there are self-healing mechanisms in place that prevent data loss.
DO capture transfer state and exceptions	Observability of normal and abnormal behavior in data transfer is paramount to be able to troubleshoot and prevent data loss and services availability issues.
DO use native integration when integrating solutions on the same vendor or same technology	By using technology coming from the same vendor/technology as the integration points solutions, we ensure easier maintenance upgrade, easier support and optimal performance.

DO acknowledge integration coupling	Avoid maintenance overhead and delays by understanding how often changes in the interface happen, and how much control is needed over coordinating changes on both sides (provider, consumer)
DO use asynchronous integration when there is no need to wait for a response	Avoid continuous polling for data if data changes are not predictable to improve resource utilization and decouple producers from consumers so that they don't need to be available at the same time.

Integration DON'Ts

Rule	Impact
DO NOT bypass the Integration Platform	Bypassing the Integration Platform, having point-to-point direct integrations defeats the whole purpose of integration standardization, proper observability, cost control and modernization, also might introduce security vulnerabilities.
DO NOT integrate new solutions with sunset solutions	Avoid creating new dependencies towards sunset solutions, which makes decommissioning extremely difficult and costly. Instead, select alternative successor solutions.
DO NOT use analytical data (MDW, DANA), for transactional use cases	DANA should be used as data source ONLY for analytical use cases: reporting, visualizations, statistics, aggregations, historical data analysis
DO NOT use file transfer mechanism	Historically, we have been experiencing issues and risks generated by file-based transfer: • security: weak level of encryption, authentication, and access control • scalability: limited concurrency due to data corruption risks • reliability: difficult to do error handling and data validation due to unstructured nature of data
DO NOT use direct database connection	A solution must not connect directly to another solution's database to avoid tight coupling to database technology and

	coupling to an internal data model and security risks.
DO NOT introduce business logic in the integration layer	Business logic must be maintained in the producer and/or in the consumer solutions but not in the integration layer. Changing the business rules should not affect integration whatsoever provided the interface contract is stable. The only logic acceptable in the integration layer is conversion from source data objects to destination data objects.